

Lección 16: Sentry Performance

Performance de Usuario en Producción

Agenda

- ¿Por qué Performance = Negocio?
 - Core Web Vitals
 - Real User Monitoring (RUM)
 - Performance Budget
 - Conceptos Técnicos
 - Herramientas y Medición
-

El Costo de la Lentitud

Datos reales:

- **53% usuarios abandonan** si tarda > 3 segundos
- **100ms retraso** = 1% menos conversiones
- **Amazon:** 100ms latencia = 1% ventas anuales
- **Google:** 10→30 resultados = 20% menos tráfico

Conclusión: Performance = Revenue

Funciona vs Funciona Rápido

Funcional

=====

– 5 segundos

Performante

=====

– 1.5 segundos

- JS bloquea UI
- Imágenes sin optimizar
- Bundle 2MB
- Peticiones en cascada
- Sin métricas
- Código asíncrono
- WebP + lazy loading
- Code splitting
- Ruta crítica optimizada
- Web Vitals monitoreadas

Core Web Vitals

¿Por qué importan más?

Métricas tradicionales (“tiempo de carga”) no reflejan experiencia real:

- Página “carga” en 1s pero bloqueada 5s
- Contenido aparece pero no es interactivo 8s
- Elementos se mueven mientras usuario hace click

Core Web Vitals = Experiencia Real del Usuario

LCP: Largest Contentful Paint

¿Qué mide? Cuándo aparece el contenido principal

Qué cuenta como LCP:

- Imagen más grande visible
- Bloque de texto más grande
- Video más grande

Umbrales:

-  Excelente: < 2.5s
-  Necesita mejora: 2.5 - 4.0s
-  Malo: > 4.0s

LCP: Ejemplo

Mal optimizado:

0s: Click
1s: HTML descargado
2s: CSS procesado
3s: JavaScript parseado
5s: ¡Imagen del producto! ← LCP aquí

Optimizado:

0s: Click
0.8s: CSS crítico inline
1.2s: ¡Imagen del producto! ← LCP aquí

FID: First Input Delay (Retraso de Primera Entrada)

¿Qué mide? Retraso desde que usuario interactúa hasta que navegador responde

Causas de FID alto:

- JavaScript pesado bloqueando hilo principal
- Scripts de terceros sin optimizar
- Cálculos síncronos costosos

Umbrales:

-  Excelente: < 100ms
-  Necesita mejora: 100 - 300ms
-  Malo: > 300ms

FID: Ejemplo

Usuario: Click en "Add to Cart"
→ JavaScript procesando algo pesado
→ 500ms después recién responde el click

→ Usuario frustrado: "¿Está roto?"

FID = 500ms  MALO

Solución: Code splitting, web workers, procesamiento asíncrono

CLS: Cumulative Layout Shift (Cambio Acumulativo de Diseño)

¿Qué mide? Cuánto se mueven los elementos durante la carga

¿Por qué es crítico?

1. Usuario intenta hacer click en "Comprar"
2. Banner publicitario se carga arriba
3. Todo se mueve hacia abajo
4. Usuario hace click en "Cancelar" sin querer

Umbrales:

-  Excelente: < 0.1
 -  Necesita mejora: 0.1 - 0.25
 -  Malo: > 0.25
-

Monitoreo de Usuarios Reales (RUM)

Tests sintéticos:

- Conexiones rápidas de fibra
- Dispositivos de alta gama
- Condiciones ideales
- Sin extensiones

Realidad de usuarios:

- 60% navegan desde móviles
- Conexiones 3G/4G variables

- Dispositivos con 2GB RAM
- Bloqueadores de anuncios, extensiones

RUM captura experiencia real

RUM: Datos Reales

```
// Captura automática de Sentry:
{
  device: "Samsung Galaxy A21",
  connection: "3g",
  memory: "2gb",
  lcp: 4200, // 4.2s ❌
  fid: 280, // 280ms ⚠️
  cls: 0.15, // ✅
  location: "Madrid, Spain",
  userAgent: "Chrome/120 Android",
  viewportSize: "412x915"
}
```

Presupuesto de Performance

¿Qué es? Límite que te impones para evitar que la app se vuelva lenta sin darte cuenta

Tipos de presupuestos:

Basado en tiempo:

- “LCP < 2.5s”
- “FID < 100ms”

Basado en tamaño:

- “Bundle principal < 250KB”
- “Imágenes totales < 1MB”

Basado en peticiones:

- “Máximo 10 peticiones críticas”

- “Máximo 3 fuentes externas”
-

Optimización de Ruta Crítica

No optimizado:

1. HTML (20KB) – 200ms
 2. CSS (50KB) – 300ms
 3. Google Fonts (30KB) – 400ms
 4. JS main (200KB) – 800ms
 5. Llamada API – 500ms
 6. Imágenes – 600ms
- Total: 2.8s hasta ver productos

Optimizado:

1. HTML + CSS crítico inline (25KB) – 250ms
 2. JS crítico (50KB) – 300ms
 3. Llamada API (en paralelo) – 500ms
- Total: 500ms hasta ver productos
-

¿Qué es un Bundle?

Bundle: Archivo único que contiene todo tu código JavaScript

Proceso de bundling:

Código fuente (múltiples archivos):

- src/App.tsx
- src/components/Header.tsx
- src/components/Footer.tsx
- src/utils/helpers.ts
- node_modules/react
- node_modules/lodash
- ... (100+ archivos)

↓ Herramienta de build (Vite, Webpack)

Bundle final:

- main.js (500KB) ← TODO en 1 archivo

Problema: Usuario descarga TODO, aunque solo use el 20%

Code Splitting

Bundle monolítico:

- Usuario entra a /login
- Descarga 500KB incluyendo:
 - Código de checkout (no lo necesita)
 - Dashboard admin (no es admin)
 - Librerías de gráficos (no hay gráficos)

Con code splitting:

- Usuario entra a /login
- Descarga 80KB específico para login
 - Si va a checkout, descarga ese chunk
 - Solo paga el costo de lo que usa
-

Code Splitting: Implementación

```
// Route-based splitting
const LoginPage = lazy(() => import('./pages/LoginPage'))
const Dashboard = lazy(() => import('./pages/Dashboard'))

// Feature-based splitting
const ChartLibrary = lazy(() => import('./components/Charts'))

// Third-party splitting
const DatePicker = lazy(() => import('react-datepicker'))
```

Sugerencias de Recursos

Tipos de sugerencias:

preload - “Necesitarás esto pronto”

```
<link rel="preload" href="/api/products" as="fetch" />
<link rel="preload" href="hero.webp" as="image" />
```

prefetch - “Puede que necesites después”

```
<link rel="prefetch" href="/dashboard.js" />
```

preconnect - “Vas a usar este dominio”

```
<link rel="preconnect" href="https://fonts.googleapis.com" />
```

Optimización de Imágenes

Problema típico:

Imagen: 4MB PNG
Mostrada: 300x200px
→ Usuario descarga 4MB
→ En 3G = 8 segundos

Solución moderna:

```
<picture>
  <source media="(max-width: 600px)" srcset="product-mobile.webp 300w" />
  <source media="(min-width: 601px)" srcset="product-desktop.webp 800w" />
  
</picture>
```

WebP: 25-35% más pequeño que JPEG

¿Qué es la Hidratación?

Hydration (Hidratación): Proceso donde JavaScript “despierta” el HTML estático

Proceso visual:

1. SERVER envía HTML estático

`<button>Click</button>`

← Usuario lo VE
pero NO funciona

2. BROWSER descarga JavaScript
↓ Descargando JS...

3. HYDRATION: JS se "conecta" al HTML

`<button onClick={...}>`

← Ahora SÍ funciona

Por qué importa para performance:

- Usuario VE contenido → LCP puede ser bueno
- Pero NO puede interactuar hasta hidratación → FID/TTI malos
- Hidratación pesada = página “congelada”

Métricas Avanzadas

Más allá de Core Web Vitals, estas métricas dan visibilidad profunda:

1. **TTI (Time to Interactive)** - Tiempo hasta Interactividad
2. **TBT (Total Blocking Time)** - Tiempo Total de Bloqueo
3. **Speed Index** - Índice de Velocidad

TTI: Time to Interactive

¿Qué mide? Cuándo la página está completamente interactiva

Criterios para TTI:

- ✅ Contenido útil renderizado (First Contentful Paint)

- ✓ Manejadores de eventos registrados para elementos visibles
- ✓ Página responde a interacciones en < 50ms

Ejemplo:

0s: Página carga
2s: Contenido visible
3s: JavaScript todavía ejecutándose (página no responde)
5s: ✓ TTI – Ahora SÍ responde a clicks

Umbrales: ✓ < 3.8s | ⚠ 3.9-7.3s | ✗ > 7.3s

TBT: Total Blocking Time

¿Qué mide? Tiempo total donde el hilo principal estuvo bloqueado

Cómo funciona:

Task 1: 30ms (< 50ms) → No cuenta
Task 2: 80ms (> 50ms) → Bloqueo: 30ms (80 – 50)
Task 3: 150ms (> 50ms) → Bloqueo: 100ms (150 – 50)
Task 4: 40ms (< 50ms) → No cuenta

TBT = 30ms + 100ms = 130ms

Causas comunes:

- JavaScript pesado sin code splitting
- Scripts de terceros bloqueantes
- Procesamiento síncrono costoso

Umbrales: ✓ < 200ms | ⚠ 200-600ms | ✗ > 600ms

Speed Index

¿Qué mide? Qué tan rápido el contenido se hace visualmente completo

Lento (Speed Index: 4000):

1s: 10% visible
2s: 20% visible
3s: 50% visible
4s: 100% visible

Rápido (Speed Index: 1200):

0.5s: 60% visible
1s: 90% visible
1.2s: 100% visible

Por qué importa:

- Percepción del usuario de velocidad
- Mejor que “tiempo de carga” tradicional
- Captura experiencia visual

Umbrales:  < 3.4s |  3.4-5.8s |  > 5.8s

Comparación de Métricas

Escenario: Página de producto

Métrica	Valor	Diagnóstico
=====		
LCP	2.1s	 Imagen del producto carga rápido
FID	120ms	 JavaScript bloquea interacción
CLS	0.05	 Layout estable
TTI	6.2s	 Tarda mucho en ser interactivo
TBT	450ms	 Mucho procesamiento bloqueante
Speed Index	2.8s	 Se puebla visualmente rápido

Acción requerida: Optimizar JavaScript (afecta FID, TTI, TBT)

Ejercicio 1: Performance Transaction Monitoring

Prompt:

Actúa como un performance engineer instrumentando código con Sentry transac

CONTEXT0: Performance = Revenue: 100ms retraso = 1% menos conversiones (dat Amazon). Real User Monitoring (RUM): mide performance en producción con usu reales (NO sintético). Sentry Transactions: miden timing de operaciones específicas (funciones, API calls, renders). Transaction name: identificado único ('**calculate-total**', '**checkout-flow**'). Op (operation **type**): categoría ('**function**', '**http.client**', '**db.query**'). transaction.finish(): marca fin y datos a Sentry. Performance dashboard: muestra average, P95 (percentil 95), P95: 95% de requests son más rápidos que este valor (útil para SLAs).

TAREA: Instrumenta calculateTotal con Sentry transaction para medir perform

FUNCIÓN A INSTRUMENTAR:

- Archivo: src/shared/utils/cart-operations.ts (o similar)
- Función: calculateTotal(items: CartItem[]): number
- Operación: Suma price × quantity de todos los items

IMPLEMENTACIÓN:

- Import: import * as Sentry from '**@sentry/react**'

TRANSACTION PATTERN:

```
export function calculateTotal(items: CartItem[]) {
  const transaction = Sentry.startTransaction({
    name: 'calculate-total',
    op: 'function'
  })

  // Lógica existente (reduce sum)
  const total = items.reduce(...)

  transaction.finish() // Marca fin + envía timing
  return total
}
```

PROPS TRANSACTION:

- name: Identificador único (aparece en dashboard)
- op: '**function**' (categoría de operación)

VALIDACIÓN EN SENTRY DASHBOARD:

1. Ejecutar app + agregar items al carrito (llama calculateTotal)
2. Ir a sentry.io → Performance → Transactions
3. Buscar transaction "**calculate-total**"
4. Verificar:

- Average duration (promedio): ~2-5ms esperado
- P95: < 10ms ideal
- Throughput: # de llamadas por minuto

VALIDACIÓN: Transaction "calculate-total" debe aparecer con timing < 10ms P

Aprende: Transacciones de performance revelan cuellos de botella en producción con usuarios reales

Ejercicio 2: Métricas de Negocio Personalizadas

Prompt:

Actúa como un analista de producto instrumentando métricas de negocio con S

CONTEXT0: Métricas personalizadas capturan KPIs de negocio (NO solo perform Ejemplos de métricas de negocio: promedio de tamaño del carrito, pasos del tasa de adopción de features. API Sentry.metrics: `set` (gauge - valor actual (counter), distribution (histogram). Tags: dimensiones para filtrar/agrupar (category: `'business'` vs `'technical'`). Unit: unidad de medida (`'items'`, `'milliseconds'`, `'bytes'`). Dashboard de métricas: analiza tendencias, promedio a través del tiempo. Separación de responsabilidades: métricas de negocio s técnicas (facilita análisis por stakeholder).

TAREA: Agrega métrica personalizada para rastrear tamaño promedio del carri

UBICACIÓN:

- Archivo: `src/context/CartContext.tsx` (o donde se maneja cart state)
- Función: `addItem` (o cuando `items[]` cambia)

IMPLEMENTACIÓN:

- Import: `import * as Sentry from '@sentry/react'`
- Timing: DESPUÉS de actualizar state (usar `newItems.length`, NO `items.lengt`

METRIC PATTERN:

```
const addItem = (product: Product, quantity: number) => {
```

```
const newItems = [...items, { ...product, quantity }]
setItems(newItems)

// Business metric
Sentry.metrics.set('cart.items.count', newItems.length, {
  tags: { category: 'business' },
  unit: 'items'
})
}
```

SENTRY.METRICS.SET PARAMS:

- Param 1: Metric name ('cart.items.count')
- Param 2: Value (newItems.length)
- Param 3: Options object:
 - tags: { category: 'business' } (para filtrar en dashboard)
 - unit: 'items' (claridad en dashboard)

VALIDACIÓN EN DASHBOARD DE SENTRY:

1. Agregar varios items al carrito (3-5 items)
2. Ir a sentry.io → Metrics → Custom Metrics
3. Buscar métrica "cart.items.count"
4. Verificar:
 - Promedio: debe mostrar ~3-5 items
 - Tag category: 'business' visible
 - Unidad: items
 - Gráfico de tendencia a través del tiempo

VALIDACIÓN: Métrica "cart.items.count" debe aparecer con promedio ~3-5 item

Aprende: Métricas de negocio personalizadas vinculan

observabilidad técnica con KPIs de producto

Puntos Clave

1. **Performance = Negocio:** 100ms = 1% conversión
2. **Core Web Vitals:** LCP, FID, CLS
3. **RUM:** Mide usuarios reales, no sintéticos
4. **Presupuesto de Performance:** Previene regresiones
5. **Ruta Crítica:** Optimiza lo crítico primero

6. **Code Splitting:** Carga solo lo necesario
7. **Optimización de Imágenes:** WebP + lazy loading